

Local-first code intelligence graph.

1. The Truth About Code-Review-Graph / Graphify

It is **not** an MCP Host. It is a highly specialized **MCP Server (a Tool)**.

- **What it does:** It crawls your massive folder of forked repositories (Fabric, Open Interpreter, Firecrawl, and the MarkTechpost tutorials) and builds a local, persistent dependency map (a graph database) of how all that code connects.
- **Why it says "for MCP and CLI":** It exposes that graph data via the Model Context Protocol standard. When an agent asks it a question, it doesn't dump the whole repository. It looks at the graph and says, *"Only these 3 lines of code in file X matter to the bug in file Y."* That is where your **context reductions** come from.

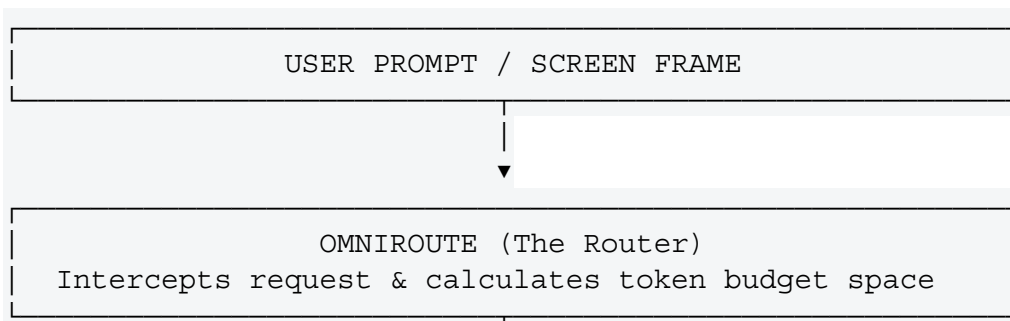
2. Where OmniRoute Fits (The Router vs. The Map)

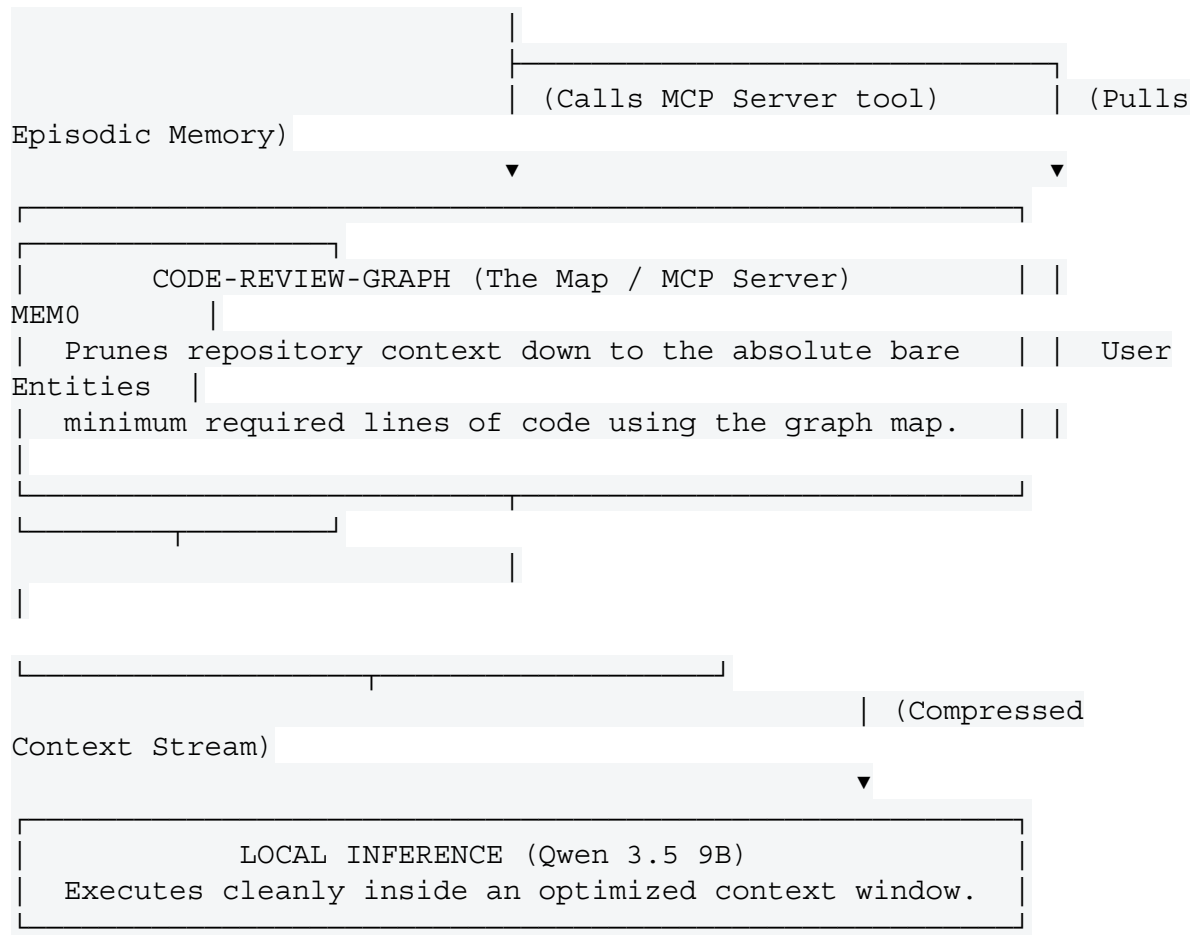
If code-review-graph is the **Map** of your code, **OmniRoute** is the **Traffic Controller** sitting in front of your models.

OmniRoute does not map your files. OmniRoute handles **Context Compression and Token Routing**. When you speak a prompt or pass a screen frame into your Qwen 3.5 models, OmniRoute intercepts the prompt and decides *where* to get the missing information to fill it out.

3. The On-Device Handshake (How You Start Here)

Because you want to wire **OmniRoute first**, here is exactly how it pulls data from your code-review-graph to feed your local NPU models:





How You Set This Up (The Flow)

1. **Run the Graph Extractor:** You run code-review-graph locally in Termux over your repository storage. It indexes everything once and sits quietly as a local background daemon.
2. **Initialize OmniRoute:** You boot OmniRoute as the local proxy endpoint.
3. **The Context Compression Loop:** When you ask your Qwen 3.5 assistant to figure out a script using a tutorial from your MarkTechpost folder:
 - o **OmniRoute** catches the prompt.
 - o OmniRoute queries the code-review-graph local MCP tool interface to ask for the exact code references.
 - o code-review-graph returns *only* the specific snippets that matter, stripping out the rest of the file boilerplate.
 - o OmniRoute packs those tight code references alongside your **Mem0** entities, compresses the text tokens, and hands the final micro-payload to your NPU.

This is exactly how you achieve those massive **benchmarked context reductions** on your local device without running out of RAM on your phone's chip.

Since we are keeping it completely authentic to the repos you have: Do you want to write out the structural configuration block that tells OmniRoute how to call this local code graph, or do

you want to map out how it tags the resulting execution tracks for your **Reasoning Bank** first?